HinglishVinglish plan

HinglishVinglish

Analysis & Implementation Plan

Snapshot of October 26, 2026

Plan overview · Linguistics & HV-Roman spec · Learning science · Technical architecture · Development plan · Data & licensing

HinglishVinglish: Analysis & Implementation Plan

Oct 26, 2026 · @Ayan

Summary

The premise works, but only if the app treats Devanagari as the hidden source of truth and romanization as a presentation layer with one taught spelling plus a set of accepted variants. Today the MVP does exact sentence lookup on raw text with ITRANS, which will miss almost every real input.

Headline recommendations:

1. **Pivot through Devanagari.** Every Hindi word is stored once in Devanagari. Romanized forms (canonical and variants) are keys that resolve to it, never the stored truth.
2. **Define a house romanization, "HV-Roman", not raw ITRANS.** All-lowercase ASCII, Hindi-style schwa deletion (raam, not raama), doubled vowels for long sounds, with an optional diacritic learner view. ITRANS stays internal as a lossless pivot.
3. **Build a variant lexicon from Dakshina and Aksharantar.** Map attested spellings (bahut, bohot, bhot) to one Devanagari entry, with a phonetic-key fallback for unseen spellings.
4. **Replace exact sentence matching with a layered translator:** exact phrase memory, then a neural model (IndicTrans2) for anything else, with word-by-word gloss for learners.
5. **Build the learning app on the same lexicon:** spaced repetition (FSRS), typing drills that accept any listed variant but teach the canonical one, and script-reveal exercises that move learners toward Devanagari.
6. **Check licensing before shipping.** Several key corpora are research-only or non-commercial.

Decisions (Oct 26, 2026)

Question	Decision	What it changes
Commercial or not?	Functionality first; commercial later	Any source may be used in development, but every row is license-tagged so a commercial build can exclude non-commercial data
Delivery surface	Web and mobile app	One Expo (React Native) codebase for iOS, Android and web; FastAPI backend
Final-vowel convention	kya / achha	Word-final आ, ई, ऊ are written a, i, u; medial ones aa, ee, oo
Audio	Start with whatever works, upgrade later	Pre-generated TTS (Indic Parler-TTS) first; native recordings for core words later
Scope	Hindi first; eventually all non-Latin-script languages	Engine is built around a pluggable LanguagePack so other scripts slot in

How this plan is organized

This tab is the overview. The detail a developer needs lives in five tabs:

- Linguistics & HV-Roman spec — the normative romanization spec, phonology, normalization, phonetic key, grammar and ambiguity rules
- Learning science — the research behind every learning feature, the learner model and the curriculum
- Technical architecture — stack, system diagram, interfaces, API, budgets and multi-script design
- Development plan — ticket-level backlog with acceptance criteria and the testing strategy
- Data & licensing — provenance and license of every source, and the build-time license gate

Analysis of the current MVP

The MVP's architecture (CLI, translator, dataset manager) is a sensible start, but its two core design choices, raw ITRANS and exact sentence matching, work against the premise. A learner typing "bahut accha" or "hello" will almost always get "No translation found."

Issue in README / MVP	Why it matters	Fix in this plan
ITRANS is case-sensitive (A, I, U, T, D, N, Sh are distinct letters)	English speakers type lowercase; "Tota" and "tota" mean different consonants. It is not "easy for English speakers" in practice.	Learner-facing HV-Roman is all lowercase; ITRANS kept only as an internal lossless pivot
README examples don't match ITRANS output	धन्यवाद in ITRANS is dhanyavAda, not dhanyavaad. The doc already mixes two schemes.	One documented canonical scheme, with generated examples tested in CI
No schwa deletion	Script-to-Roman tools keep the inherent final "a" (shubha, raama), but Hindi speakers say shubh, raam. Learners will mispronounce.	Schwa-deletion step in the Devanagari-to-Roman converter
Pronunciation table errors	The inherent vowel in ka/na is the "u" in "but", not the "o" in "not".	Rewrite guide with audio and IPA
Exact sentence matching only	The dataset is sentence pairs; single words and novel sentences never match.	Layered translator: phrase memory, then neural MT, then word gloss
Romanized input must equal the dataset's exact transliteration	Any spelling variant misses. Romanized Hindi has no conventional spelling.	Variant lexicon + phonetic key resolve input to Devanagari first
No text normalization	Case, punctuation, nukta (ज़ vs ज), anusvara vs half-nasal all break lookups.	Unicode NFC + Hindi-specific normalizer on both sides
Kaggle credentials needed at runtime	High setup friction; provenance and license of the Kaggle file are not stated on its page.	Offline build step produces a versioned SQLite artifact from licensed sources
"Hinglish" in the name, but no code-mixing	Real Hinglish mixes English words into Hindi sentences.	Token-level language ID so English words pass through untouched

Research findings

The hard problem is not translation, which mature open models already do well, but romanization: there is no conventional Latin spelling for most Hindi words, so spelling varies widely (Benton et al., Google, 2025). Every resource below exists to tame that.

Spelling variation is the core problem. The same word appears as bahut, bohot, boht and bhot (dhvani). Variation comes from vowel omission, consonant simplification, dropped aspiration and non-standard spellings (Modeling Romanized Hindi, 2025; Parikh & Solorio, 2021). Prior work normalizes with context clustering, phonetics or IPA as a bridge.

Schwa deletion decides how a word should be spelled and said. Hindi drops the inherent vowel at word ends and in some medial positions, and getting it wrong is the main error in Hindi grapheme-to-phoneme conversion (Wikipedia: Hindustani orthography). A statistical classifier outperforms rule-based approaches, with code and data released (Arora et al., ACL 2020; repo).

Resources

Resource	What it gives HinglishVinglish	License / caveat
Dakshina (Google)	Hindi romanization lexicon: 25,000 train + 5,000 dev/test Devanagari word types, ~1.8 attested romanizations each, plus 10,000 hand-romanized sentences	CC BY-SA 4.0 (attribution + share-alike)
Aksharantar (AI4Bharat)	26M transliteration pairs across 21 languages; the largest variant source	CC-BY (manual data) / CC0 (mined data)
IndicXlit	Roman-to-Devanagari model returning ranked candidates; ~11M params; a CTranslate2 build avoids the Fairseq install and adds a reverse model	Open-source
IndicTrans2	Open English↔Hindi NMT across 22 languages; distilled 200M models; INT8 ONNX builds run offline in ~300 MB	Checkpoints MIT
indic_transliteration	Rule-based Devanagari↔ITRANS/IAST/HK; OPTITRANS has a "lay Indian" output (kRShNa → krishna)	Open-source; already a dependency
IIT Bombay corpus	1.49M English–Hindi sentence pairs plus Hindi–English WordNet dictionary entries	Non-commercial research only

dhvani	Off-the-shelf Hinglish phonetic normalizer via IPA	Useful as a baseline to compare against
py-fsrs	Modern spaced-repetition scheduler (difficulty, stability, retrievability) with a parameter optimizer	MIT

Romanization schemes considered. ITRANS, Harvard-Kyoto, IAST, SLP1 and Velthuis are all supported by indic_transliteration. They are lossless but designed for Sanskrit: they keep every schwa and rely on case or diacritics. None matches how Hindi speakers actually type, which is why a house scheme is needed.

System design: standardized romanization and bidirectional translation

Every Hindi word lives once, in Devanagari; each has exactly one canonical HV-Roman spelling the app teaches and any number of accepted variants it understands. Translation always happens Devanagari↔English, so the romanization layer and the translation layer can improve independently.

1. The canonical scheme: HV-Roman

HV-Roman is lowercase ASCII, matches how Hindi speakers already type, and is generated by rule from Devanagari, so it is consistent by construction. Where it is lossy (त vs ट), the lexicon disambiguates, and a learner "precise view" shows IAST diacritics (ṭ, ā).

Sound / letter	HV-Roman	Example	Accepted variants
अ short a	a (dropped where Hindi deletes schwa)	राम raam	ram
आ long ā	aa medially, a word-finally	राजा raaja, क्या kya	a, aa everywhere
इ / ई	i / ee	दिल dil, ठीक theek	ii, i
उ / ऊ	u / oo	तुम tum, दूध doodh	uu, u
ए / ऐ	e / ai	है hai	he, h
ओ / औ	o / au	और aur	or
aspirates	kh gh chh jh th dh ph bh	अच्छा achha	accha, acha
च / छ	ch / chh	चाय chaay	chai, chay
श ष / स	sh / s	शुभ shubh	subh
व	v	वाला vaala	wala, vala
nukta ज़ फ़ ड़	z f d	ज़रूर zaroor	jaroor, zarur
ं ँ nasal	n (m before p/b/m)	नहीं nahin	nahi, nai

The schwa rule is the single most important step: शुभ is taught as shubh, not ITRANS shubha. Rule-based schwa deletion handles most words; the Arora et al. classifier handles the rest, and the lexicon can override any word by hand.

2. The variant lexicon

Each entry links a Devanagari word to its canonical spelling, English glosses and a ranked list of accepted variants. Variants come from four sources, in order of trust: hand-curated overrides, Dakshina's human-attested romanizations, Aksharantar's mined pairs, and rule-generated alternations (aa↔a, ee↔i, oo↔u, w↔v, dropped aspiration h, z↔j, f↔ph, ai↔e, final n dropped). Each variant stores a frequency score so common spellings rank first.

For spellings never seen, a phonetic key collapses what romanizers vary on (vowel length, aspiration h, doubled letters, w/v, z/j). bahut, bohot and bhot share a key; candidates sharing it are reranked by weighted edit distance and word frequency.

3. Romanized Hindi → English

1. **Normalize and tokenize:** lowercase, strip punctuation, keep numerals.
2. **Tag each token Hindi or English** with an English wordlist plus a small character n-gram classifier, so code-mixed "mera phone kharab hai" keeps "phone".
3. **Resolve each Hindi token to Devanagari candidates:** canonical match, then variant match, then phonetic key, then IndicXlit top-k as a fallback.
4. **Pick the best sentence** by rescoring candidate combinations with a Hindi n-gram language model (KenLM on Hindi Wikipedia), so "ki" resolves to कि or की by context.
5. **Translate** the Devanagari sentence: exact phrase memory first, IndicTrans2 hi→en otherwise.
6. **Return** English, Devanagari, the canonical HV-Roman respelling ("you wrote bohot, we teach bahut") and a word-by-word gloss.

4. English → Romanized Hindi

1. **Phrase memory** for curated, learner-friendly phrases (greetings, survival phrases), which beat MT on register.
2. **IndicTrans2 en→hi** for everything else, producing Devanagari.

3. **Devanagari → HV-Roman**: lexicon lookup for each word's canonical spelling; the rule converter with schwa deletion for unknown words.
4. **Gloss** each Hindi word with its lexicon meaning, plus a formality note (tum / aap / tu) where relevant.

Invariant tested in CI: for every lexicon word, resolving its canonical HV-Roman spelling returns the same Devanagari word. This is the guarantee that the standard is truly bidirectional.

Language learning application

The learning app reuses the translation engine as its grader: a typed answer is correct when it resolves to the target Devanagari word, so learners are never failed for a valid variant spelling, but are always shown the canonical one.

Exercise types

Exercise	Prompt → answer	What it trains
Recognize	HV-Roman (+ audio) → pick or type English	Comprehension
Produce	English → type Hindi in any accepted spelling	Recall and spelling
Listen	Audio only → type Hindi	Ear for schwa, aspiration, retroflex sounds
Read script	Devanagari → type HV-Roman or English	Script literacy
Cloze	Sentence with a gap → fill the word	Grammar in context (postpositions, gender agreement)
Translate	Short sentence either way, graded by the pipeline	Putting it together

Automatic grading mapped to spaced repetition

Each word is an FSRS card, and grades come from the resolver rather than self-rating, which removes the honesty problem of flashcard apps.

- Exact canonical spelling, fast answer → **Easy**
- Accepted variant (bohot for bahut) → **Good**, plus a gentle "standard spelling: bahut" note
- Resolves to the right word only via phonetic key or a near-miss edit → **Hard**, with a diff of the spelling
- Resolves to a different word, or nothing → **Again**

Review logs feed the py-fsrs optimizer, so intervals adapt to each learner after a few hundred reviews.

Curriculum

1. **Core vocabulary first:** the 1,000 most frequent Hindi words (ranked from a Hindi corpus), grouped into themed units: greetings, numbers, family, food, travel, time, everyday verbs.
2. **Phrases before grammar:** survival phrases from the curated phrase memory, then sentence patterns that teach hai/hoon/ho, postpositions (ka/ki/ke, mein, par) and verb gender.
3. **Script bridge in four stages:** Roman only, Roman with Devanagari beneath, Devanagari with Roman on tap, Devanagari only. Learners advance per letter group as their reading accuracy passes a threshold.
4. **Pronunciation focus:** minimal pairs for dental vs retroflex (त/ट) and plain vs aspirated (क/ख), which HV-Roman's lowercase form hides.

Translator as a study tool

Any sentence translated in the app can be saved to a personal deck with one action. The output shows the word-by-word gloss and the respelling note, so the translator teaches instead of only answering.

Data model and project structure

A single SQLite file, built offline and versioned, holds the lexicon, variants, phrase memory and learner state; models load lazily so the app starts without them.

```
-- Lexicon: one row per Hindi word (Devanagari is the key)
CREATE TABLE lemma (
  id INTEGER PRIMARY KEY,
  deva TEXT UNIQUE NOT NULL,        -- नमस्ते
  canonical TEXT NOT NULL,          -- HV-Roman: namaste
  precise TEXT,                     -- IAST: namaste
  pos TEXT, gender TEXT,
  freq_rank INTEGER,
  override BOOLEAN DEFAULT 0        -- hand-curated spelling wins
);
```

```
        CREATE TABLE gloss (
          lemma_id INTEGER REFERENCES lemma(id),
          english TEXT NOT NULL, sense_rank INTEGER
        );
        CREATE TABLE variant (
          spelling TEXT NOT NULL,           -- bohot
          lemma_id INTEGER REFERENCES lemma(id),
          source TEXT,                      -- curated | dakshina | aksharantar | rule
          score REAL,
          PRIMARY KEY (spelling, lemma_id)
        );
        CREATE TABLE phonetic_key (key TEXT, lemma_id INTEGER);
        CREATE TABLE phrase (
          id INTEGER PRIMARY KEY,
          english TEXT, deva TEXT, canonical TEXT,
          register TEXT, unit TEXT, source TEXT
        );
        -- Learner state
        CREATE TABLE card (
          id INTEGER PRIMARY KEY, user_id INTEGER,
          lemma_id INTEGER, phrase_id INTEGER,
          exercise TEXT, fsrs_state TEXT, due TIMESTAMP
        );
        CREATE TABLE review_log (
          card_id INTEGER, rating INTEGER, answer TEXT,
          resolved_as TEXT, latency_ms INTEGER, reviewed_at TIMESTAMP
        );
```

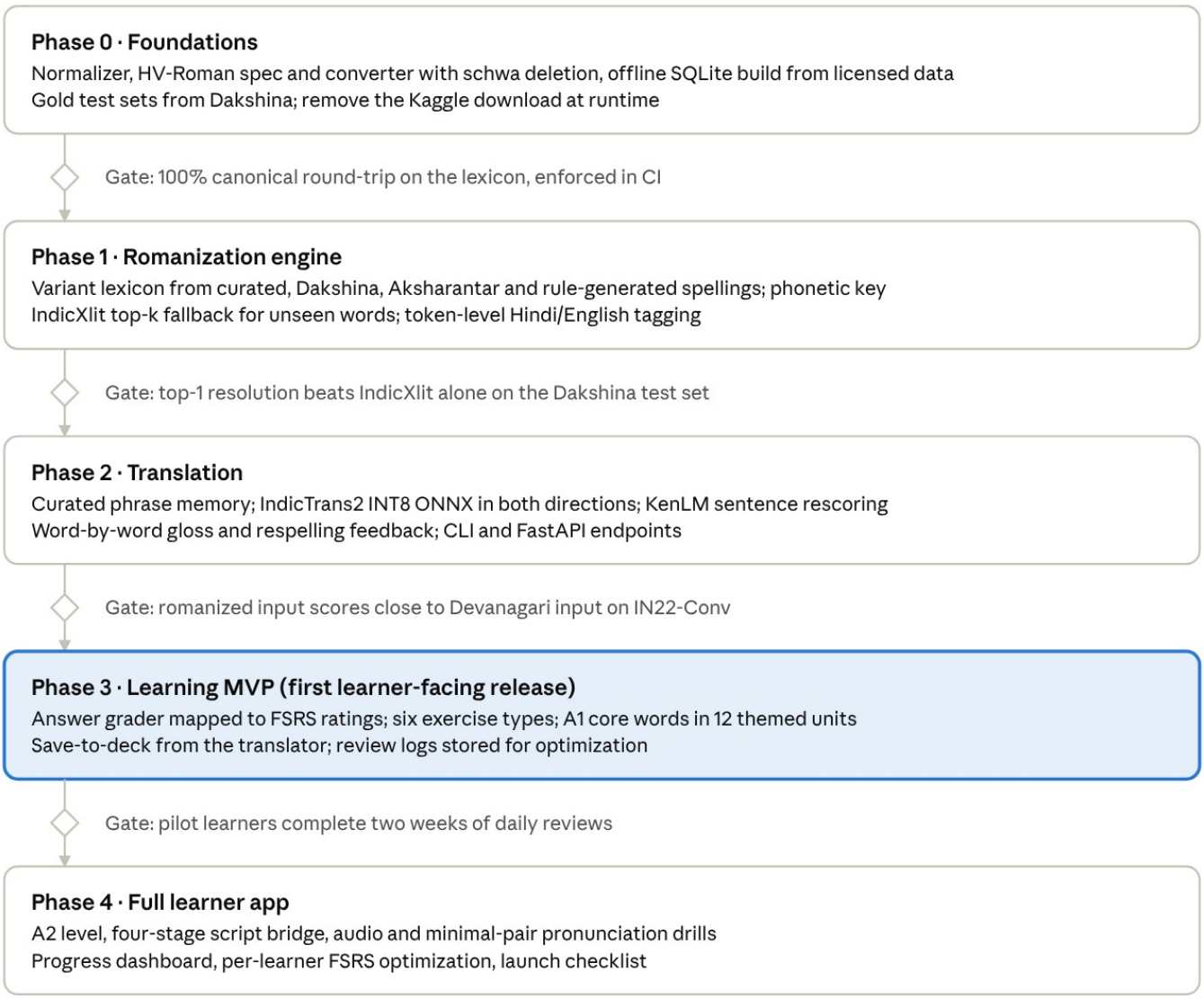
```
hinglishvinglish/
├── hv/
│   ├── romanize/
│   │   ├── scheme.py        # HV-Roman spec: Devanagari -> canonical
│   │   ├── schwa.py         # rules + classifier
│   │   ├── normalize.py     # NFC, nukta, nasal, case
│   │   └── phonetic_key.py
│   ├── resolve/
│   │   ├── langid.py        # token Hindi/English
│   │   ├── candidates.py    # lexicon, key, IndicXlit
│   │   └── rerank.py        # KenLM sentence choice
│   ├── translate/
│   │   ├── memory.py        # phrase memory
│   │   ├── nmt.py           # IndicTrans2 (ONNX)
│   │   └── gloss.py
│   ├── learn/
│   │   ├── grader.py        # answer -> FSRS rating
│   │   ├── scheduler.py     # py-fsrs wrapper
│   │   └── curriculum.py
│   └── store.py             # SQLite access
├── build/                   # offline data pipeline
│   ├── ingest_dakshina.py
│   ├── ingest_aksharantar.py
│   ├── build_lexicon.py
│   └── build_phrases.py
├── api/                     # FastAPI service
├── web/                     # learner front end (PWA)
├── cli.py                   # successor to app.py
└── tests/                   # round-trip, gold sets
```

This layout is a sketch; the monorepo layout in Development plan supersedes it.

Phased roadmap

Build the romanization engine before touching translation or learning features, because both depend on it. Each phase ships behind a measurable gate, and Phase 3 is the first release learners see.

Five phases, each released only after a measurable check



Phases 0–2 can run with one developer; Phase 4 needs front-end and audio work. With one engine developer and one app developer, the estimate is roughly 25–27 weeks from Phase 0 to public launch; ticket-level detail is in the Development plan tab.

Evaluation, risks and decisions

The system is only as trustworthy as its gold sets, so evaluation is built in Phase 1, before any model work. Targets below are proposed starting points to tune once baselines are measured.

How it will be measured

Check	Data	Proposed bar
Canonical round-trip (HV-Roman → Devanagari → HV-Roman)	Every lexicon entry	100%, enforced in CI
Variant resolution, top-1 Devanagari	Dakshina Hindi dev/test romanizations	Beat IndicXlit alone on the same set
Sentence resolution	Dakshina's 10,000 hand-romanized sentences	Word accuracy tracked per release
Translation quality	IndicTrans2's IN22-Conv benchmark, plus a romanized copy of it	chrF++ within a few points of feeding Devanagari directly
Learner outcomes	Review logs	Retention at review near the FSRS target; day-7

		return rate

Risks

Risk	Mitigation
Licensing: the IIT Bombay corpus is non-commercial; the Kaggle file's source and license are not stated; Dakshina is CC BY-SA 4.0 (share-alike)	Keep a data-provenance table; ship only commercially clear data in any paid build; replace the Kaggle file in Phase 1
Canonical spellings are contested (achha vs accha)	Pick by attested frequency, document every choice, accept all variants as correct
Lossy lowercase hides sounds (त/ट, द/ड)	Devanagari pivot preserves them; precise view and minimal-pair drills teach them
Ambiguous short words (ki = कि / की, hai / he)	Sentence-level language-model rescoring; show alternatives when confidence is low
Model size and setup (IndicTrans2, IndicXlit)	INT8 ONNX and CTranslate2 builds; run server-side for the web app
MT register mismatches for learners (formal vs casual)	Curated phrase memory takes precedence; register tags on phrases

Decisions

- ☒ Commercial or non-commercial: functionality first, commercial later (see Decisions in the Summary).
- ☒ Delivery surface: web and mobile app.
- ☒ Final-vowel convention: kya / achha.
- ☒ Audio source: TTS first, native recordings later.
- ☒ Scope: Hindi first, built to extend to other non-Latin scripts.
- ☐ Open (Oct 26, from Phase 0 data): medial long vowels. HV-Roman 1.0 writes aa / ee / oo (raaja, theek, doodh), but only about 20% of real spellings in Dakshina do. Switching to single vowels (raja, thik, dudh) would raise agreement with people's most common spelling from 34% to 53%, at the cost of hiding vowel length outside the precise view. A change is a major scheme bump, so decide before Phase 3 content is written. Details: eval/README.md.

Sources

- Benton et al. (Google), Improving Informally Romanized Language Identification
- Roark et al., The Dakshina Dataset (LREC 2020)
- Madhani et al., Aksharantar and IndicXlit
- AI4Bharat IndicXlit repository
- IndicXlit CTranslate2 build (Hugging Face)
- AI4Bharat IndicTrans2 repository
- IndicTrans2 INT8 ONNX packages
- Kunchukuttan et al., IIT Bombay English-Hindi Parallel Corpus
- Modeling Romanized Hindi and Bengali (2025)
- Parikh & Solorio, Normalization and Back-Transliteration for Code-Switched Data
- dhvani: phonetic normalization for Hinglish
- Arora et al., Supervised Grapheme-to-Phoneme Conversion of Orthographic Schwas
- Hindustani orthography (Wikipedia)
- indic_transliteration (Python)
- py-fsrs and FSRS algorithm
- Kaggle English-Hindi dataset page
- Schwa deletion in Indo-Aryan languages (Wikipedia)
- Narasimhan, Sproat & Kiraz, Schwa-deletion in Hindi TTS (2004)
- Nisaba script processing library
- Unicode list: Devanagari nukta composition exclusions
- Aksharantar dataset card (Hugging Face)
- IndicLID / Bhasha-Abhijnaanam
- ai4bharat-transliteration (PyPI)
- IndicTransToolkit
- Indic Parler-TTS
- indictrans2-conv: note on male-perspective output
- wordfreq
- uroman
- Cepeda et al., Distributed practice meta-analysis (2006)
- Roediger & Karpicke, Reflections on the testing effect (2018)
- Kornell & Bjork interleaving, via Birnbaum et al. (2013)
- Brunmair & Richter, Interleaving meta-analysis (2019)

- Nation, How large a vocabulary (2006)
- Duolingo half-life regression
- Pruitt, Training English speakers on Hindi dental/retroflex (1993)
- Earle et al., Overnight consolidation of non-native speech sounds (2025)
- FSI language difficulty rankings

Linguistics and HV-Roman specification

This tab is the normative spec: if code and this tab disagree, the tab wins until it is changed by review.

1. The Hindi sound system, for implementers

Hindi distinguishes four things English does not, and HV-Roman's lowercase ASCII deliberately flattens two of them. The Devanagari pivot keeps them; the learning app must teach them explicitly.

Consonants

Stops and affricates form a 5 × 5 grid: five places of articulation, each with four manners plus a nasal. Rows are place, columns are manner.

Place	Voiceless	Voiceless aspirated	Voiced	Voiced aspirated (breathy)	Nasal
Velar	क k	ख kh	ग g	घ gh	ङ n
Palatal	च ch	छ chh	ज j	झ jh	ञ n
Retroflex	ट t	ठ th	ड d	ढ dh	ण n
Dental	त t	थ th	द d	ध dh	न n
Labial	प p	फ ph	ब b	भ bh	म m

Other consonants: य y, र r, ल l, व v, श sh, ष sh, स s, ह h. Flaps ड़ and ढ़ are written d and dh, as most people type them (ladka, padhna). Perso-Arabic and English loan sounds are written with a nukta dot: क़ q, ख़ kh, ग़ gh, ज़ z, फ़ f.

Vowels

Independent	Sign (matra)	Sound	HV-Roman
अ	(none, inherent)	short ə, the "u" in "but"	a
आ	◌ा	long a, "father"	aa (a word-finally)
इ	◌ि	short i, "sit"	i
ई	◌ी	long i, "see"	ee (i word-finally)
उ	◌ु	short u, "put"	u
ऊ	◌ू	long u, "boot"	oo (u word-finally)
ऋ	◌ृ	ri (Sanskrit loans)	ri
ए	◌े	long e, "they" without glide	e
ऐ	◌ै	open e, "bat" in standard Hindi	ai
ओ	◌ो	long o, "go" without glide	o
औ	◌ौ	open o, "caught"	au
ऑ	◌ॉ	English loan "o" in "doctor"	o

Any vowel can be nasalized, written with ◌ँ (chandrabindu) or ◌ं (anusvara).

The four contrasts English speakers miss

1. **Dental vs retroflex** (त/ट, द/ड). The hardest: English speakers have great difficulty perceiving it even after training, with gains of roughly 10% and little transfer to new speakers (Pruitt, 1993; Polka, 1991). HV-Roman writes both as t / d.
2. **Aspiration**, a four-way contrast (क ख ग घ). HV-Roman marks it with h, which is the letter romanizers most often drop.
3. **Vowel length** (इ/ई, उ/ऊ). HV-Roman marks it medially (ee, oo) but not word-finally.
4. **Nasalization** (है hai vs हैं hain). Marked with n, which casual typists often omit.

Rule for the product: every contrast HV-Roman flattens is still stored in Devanagari, shown in the precise view, and drilled with minimal pairs.

2. HV-Roman v1.0 mapping table

Three spellings exist for every word: **canonical** (HV-Roman, taught and displayed), **precise** (ISO 15919 diacritics, shown on tap) and **accepted variants** (understood on input, never displayed as correct). The table is the single source for the converter, its tests and the in-app guide.

Vowels and vowel signs

Devanagari	Canonical	Precise	Accepted on input
अ (inherent)	a, or nothing where schwa is deleted	a	u (as in "mujh"), omitted
आ ा	aa medially; a at word end	ā	a, aa, aah (final)
इ ि	i	i	e, ee
ई ी	ee medially; i at word end	ī	i, ii, ee, ie
उ ु	u	u	oo, o
ऊ ू	oo medially; u at word end	ū	u, uu, oo, ou
ऋ ृ	ri	ṛ	ru, r
ए े	e	ē	ae, ay, a (word-final "he" style)
ऐ ै	ai	ai	e, ae, ay
ओ ो	o	ō	oh, oo
औ ौ	au	au	o, ou, aw
ऑ ॉ	o	ô	aw, a (English spelling preferred via lexicon)

Consonants

Devanagari	Canonical	Precise	Accepted on input
क ख ग घ	k kh g gh	k kh g gh	c, q (क); k (ख, unaspirated); g (घ)
ङ	n	ṅ	ng
च छ ज झ	ch chh j jh	c ch j jh	c (च); ch, c (छ); z (ज); j (झ)
ञ	n	ñ	ny
ट ठ ड ढ	t th d dh	ṭ ṭh ḍ ḍh	t, d (unaspirated forms); dd, tt
ण	n	ṇ	nn, rn
त थ द ध	t th d dh	t th d dh	t (थ), d (ध)
न	n	n	nn
प फ ब भ	p ph b bh	p ph b bh	f (फ); b (भ), v (ब)
म	m	m	mm
य	y	y	i, e (between vowels), j
र	r	r	rr
ल	l	l	ll
व	v	v	w, u (after consonant: "swaad")
श ष	sh	ś ṣ	s, sch
स	s	s	sh, ss, c
ह	h	h	omitted (very common), hh
क़	q	q	k
ख़	kh	k̲h	k, kh
ग़	gh	ġ	g
ज़	z	z	j
फ़	f	f	ph, p
ड़	d	ṛ	r, rr, dd
ढ़	dh	ṛh	rh, r

Signs, conjuncts and symbols

Devanagari	Canonical	Notes
ं anusvara	n; m before p ph b bh m	Homorganic nasal before a consonant; vowel nasal elsewhere (हैं hain)
ँ chandrabindu	n	Same canonical as anusvara; precise view shows m̐
ः visarga	h	Rare in Hindi (दुःख duhkh)
् virama	(nothing)	Suppresses the inherent vowel
क्ष	ksh	Accept: x, ks, kch
त्र	tr	Accept: tra, thr

ज्ञ	gy	Hindi pronunciation; accept: gn, jn, gya
श्र	shr	Accept: sr
ऽ avagraha	(dropped)	Sanskrit only
। ॥ danda	.	Sentence end
०–९	0–9	Always ASCII digits
ZWJ, ZWNJ	(dropped)	Removed during normalization

Versioning rule: any change to a canonical output is a **major** HV-Roman version bump, because it changes what learners were taught. Adding accepted variants is a **minor** bump.

3. Spelling rules: Devanagari → canonical HV-Roman

The converter runs as an ordered pipeline over one normalized word; the first matching stage wins. A word in the hand-curated override list skips the pipeline entirely.

1. **Override lookup.** Curated spellings for high-frequency irregulars and English loans (डॉक्टर → doctor, फ़ोन → phone, माँ → maa).
2. **Segment into aksharas** (consonant cluster + vowel sign or inherent vowel), marking each inherent schwa as a candidate.
3. **Mark morpheme boundaries** from the lexicon (stem + suffix, compound parts). Boundaries block schwa deletion across them (Narasimhan, Sproat & Kiraz, 2004).
4. **Schwa deletion** (below).
5. **Map characters** with the table in section 2.
6. **Apply context rules:** anusvara place assimilation, word-final vowel shortening, gemination.
7. **Lowercase and strip** anything outside a–z.

Schwa deletion

The hard part of Hindi grapheme-to-phoneme conversion is deciding whether a written inherent vowel is pronounced (Arora et al., 2020).

- **Word-final:** delete the inherent schwa of the last consonant. Keep it only for words on the retained-final list (tatsama words ending in a cluster such as मित्र mitra); this list is curated, not guessed.
- **Word-medial:** apply Ohala's rule ə → ∅ / VC_CV, scanning right to left, which is reported correct about 89% of the time (Wikipedia: Schwa deletion). A nasalized vowel counts as the left-hand V (अंतराल antraal; measured on Dakshina, see eval/README.md). Never delete in the first syllable; never across a marked morpheme boundary; never next to a virama cluster.
- **Classifier override:** in Phase 1, replace the medial rule with the gradient-boosted classifier from Arora et al. (code and data), keeping the rule as fallback.

Word	Akshara analysis	Result	Why
नमक	na·ma·ka	namak	final schwa deleted; medial ma keeps its vowel (not VC_CV)
नमकीन	na·ma·kīn·na	namkeen	medial "ma" sits in VC_CV, deleted; final deleted
समझना	sa·ma·jha·nā	samajhna	morpheme boundary samajh + na keeps stem vowels as in समझ
धड़कन	dha·ṛa·ka·na	dhadkan	medial ṛa deleted, final deleted
नमस्ते	na·mas·te	namaste	schwa before a cluster is kept
धन्यवाद	dha·nya·vā·da	dhanyavaad	final deleted; medial long ā → aa

Context rules

- **Final long vowels** आ ई ऊ become a, i, u at word end (क्या kya, लड़की ladki, आलू aalu). Because final schwa is deleted, a word-final "a" always signals आ except on the retained-final list.
- **Final nasal vowels** keep their n and the vowel's medial form (हूँ hoon, कहाँ kahaan, हैं hain), with two exceptions that match common spelling: final ◌ें is ein (में mein, हमें hamein) and final ◌ीं is in (नहीं nahin).
- **Anusvara before a consonant** becomes m before p, ph, b, bh, m (संबंध sambandh) and n elsewhere (हिंदी hindi).
- **Gemination** is written doubled (पक्का pakka, बच्चा bachcha, दिल्ली dilli), with one exception decided on Oct 26: च्छ collapses to chh (अच्छा achha).
- **Aspirated + same-place aspirated** conjuncts keep both letters (बुद्ध buddh).

Every example in this section becomes a golden test case; the test file is generated from this tab's tables.

4. Normalization

Two strings that a Hindi reader would call the same word must produce the same storage form and the same match key. Storage form keeps every distinction; match key drops the ones writers routinely ignore.

Devanagari

Step	Storage form	Match key	Why
Unicode NFC	yes	yes	Precomposed nukta letters U+0958–U+095F (क़ ख़ ग़ ज़ ड़ ढ़ फ़ य़) are composition exclusions, so NFC always yields base + U+093C nukta; never compare raw strings (Unicode list discussion; codepoints.net U+0958)
Visual normalization	yes	yes	Fixes sequences that render alike but differ in code points (Nisaba's visual normalizer) (Nisaba)
Remove ZWJ / ZWNJ	yes	yes	Rendering hints only
Nukta	kept	removed	Many writers omit it (जरूर for ज़रूर)
Chandrabindu ँ vs anusvara ं	kept	merged to ं	Used interchangeably in practice
Danda, punctuation	kept as tokens	removed	
Devanagari digits	converted to ASCII	converted	

Latin input

1. Unicode NFKD, then drop combining marks, so diacritic input (ā, ṭ) folds to ASCII.
2. Lowercase.
3. Collapse any letter repeated three or more times to two (bahuuuut → bahuut), since emphasis lengthening is common.
4. Split on whitespace and punctuation; delete apostrophes inside words ("kya's" → "kyas").
5. Map digits to number tokens, translated as numbers, never transliterated.
6. Keep the original token and character offsets so the UI can highlight exactly what the learner typed.

Normalization is idempotent: normalizing twice must equal normalizing once. This is a property test.

5. Phonetic key and candidate scoring

The phonetic key is a candidate generator, not a decision: it deliberately over-merges spellings so the right word is always in the candidate list, and scoring picks among them.

Key algorithm (applied to normalized Latin input)

```
import re

DIGRAPHS = [  # applied in this order, longest first
    ("chh", "c"), ("ch", "c"), ("sh", "s"), ("ph", "f"),
    ("kh", "k"), ("gh", "g"), ("jh", "j"), ("th", "t"),
    ("dh", "d"), ("bh", "b"), ("rh", "r"),
    ("ee", "i"), ("ii", "i"), ("oo", "u"), ("uu", "u"),
    ("au", "o"), ("ou", "o"), ("aw", "o"), ("ai", "e"), ("ae", "e"),
    ("aa", "a"), ("w", "v"), ("z", "j"), ("q", "k"), ("x", "ks"),
]

def phonetic_key(word: str) -> str:
    """word must already be Latin-normalized (section 4)."""
    s = word
    for a, b in DIGRAPHS:
        s = s.replace(a, b)
    s = s[0] + s[1:].replace("h", "")       # drop non-initial h (aspiration; bahut)
    s = re.sub(r"a[ou]|o[au]", "o", s)     # vowel pairs exposed by dropping h
    s = re.sub(r"a[ie]", "e", s)
    s = re.sub(r"(.)\1+", r"\1", s)         # collapse doubled letters
    s = re.sub(r"([aeiou])n$", r"\1", s)    # drop word-final nasal n
    return s
```

Input spellings	Key	Lexicon word
bahut, bohot, bhot, boht, bahot	bot	बहुत
achha, accha, acha, achcha	aca	अच्छा
nahin, nahi, nai, nhi	ne (nhi gives ni, reached by the edit-distance fallback)	नहीं (also matches ने: resolved by scoring)
kharaab, kharab, khrab	karab, krab	ख़राब (edit distance links krab)

zaroor, jaroor, zarur, jarur	jarur	ज़रूर
theek, thik, tik	tik	ठीक
hai, he	he	है

All rows above were checked by running the function on Oct 26, 2026; they become its unit tests.

When the key misses (krab vs karab), fall back to keys within edit distance 1, then to IndicXlit.

Scoring each candidate lemma

```
        \text{score}(w) = 2.0\,\log P(\text{spelling}\mid w) + 1.0\,\log f(w) - 1.5\,d(\text{input}, v^*_w) + 1.0\,\log P_{LM}(w \mid \text{context})
```

- P(spelling | w): share of attested spellings of w that equal the input (Dakshina and Aksharantar counts, add-one smoothed); 0.5 floor for rule-generated variants.
- f(w): word frequency (wordfreq Hindi list, normalized).
- d: weighted edit distance from the input to w's closest accepted variant v*, with costs: vowel length (a/aa, i/ee) 0.2, h insertion or deletion 0.3, doubling 0.2, v/w 0.1, z/j 0.3, any other edit 1.0.
- P_LM: the sentence-level language-model term, added only in the sentence reranker.

The weights 2.0 / 1.0 / 1.5 / 1.0 are starting values. Tune them by grid search on the Dakshina Hindi dev split, maximizing top-1 accuracy, and freeze them per lexicon version.

Confidence

Report confidence as the softmax margin between the top two candidates. Below 0.6, the UI shows the alternatives ("did you mean कि ki or की ki?") instead of silently picking.

6. Grammar the engine and curriculum must model

Hindi inflects nouns, adjectives and verbs for gender, number and case, so the lexicon stores lemmas with paradigms, not just word forms. Every inflected form gets its own canonical spelling, generated from the stem so schwa deletion stays consistent.

Feature	Example	Engine requirement	Curriculum requirement
SOV word order, postpositions	main khaana khaata hoon (I food eat)	Gloss aligns Hindi order to English; never reorder the learner's Hindi	Early unit on word order
Two genders, agreement on adjectives and verbs	achha ladka / achhi ladki / achhe ladke	Lexicon stores gender; grader accepts only the agreeing form	Interleaved agreement drills
Direct vs oblique case	ladka → ladke ko, ghar → ghar mein	Paradigm table per noun class (-aa masculine, consonant-final, -i feminine)	Postpositions taught with oblique forms together
Possessive ka / ki / ke	Ram ka ghar, Ram ki gaadi	Agrees with the possessed noun, not the owner; a classic learner error	Explicit feedback message for wrong agreement
Ergative ne in perfective transitive	maine khaana khaaya	Verb agrees with the object, not the subject	Introduced at A2, after habitual and progressive tenses
Honorific pronouns tu / tum / aap	tu ja, tum jao, aap jaiye	Verb form depends on pronoun level; translator tags register	Learner chooses a default politeness level; aap taught first
Speaker gender in first person	main jaata hoon (m) / main jaati hoon (f)	English → Hindi output must use the learner's chosen gender	Profile setting asked at onboarding
Compound and light verbs	kar dena, ho jaana, kaam karna	Multi-word lexicon entries	Taught as chunks

Known MT bias. A developer fine-tuning IndicTrans2 for conversation reports that its English → Hindi output predominantly takes a male perspective (indictrans2-conv). For first-person sentences the app must regenerate or re-inflect output to the learner's gender, and test this explicitly.

Paradigm generation. Phase 1 ships hand-written paradigm rules for the three regular noun classes, regular adjectives (-aa / -i / -e), and regular verb tense forms (-ta, -ta hai, raha hai, -ega, -aa / -ya perfective). Irregular forms (gaya, kiya, diya, liya, hua) are listed explicitly in the lexicon.

7. Ambiguity classes and worked examples

Most resolution errors fall into a small number of known classes; each has a named strategy and its own test set, so regressions are visible per class.

Class	Example	Resolution
Hindi / English homographs	main (मैं / main), to (तो / to), hi (ही / hi), is (इस / is), me (में / me), log (लोग / log)	Token language ID uses neighbours: a homograph between two confident Hindi tokens is Hindi. Sentence-level LID (IndicLID's hin_Latn vs eng_Latn) breaks ties
Same key, different word	ki → कि / की; nahi → नहीं / ने	Language-model reranking; show alternatives below 0.6 confidence
Lossy canonical letters	tota → तोता; t could be त or ट	Lexicon lookup decides; rule converter is never used in the Roman → Devanagari direction
Word-final a	mitra (retained schwa) vs ladka (आ)	Retained-final list in the lexicon
Nukta omitted	jaroor for ज़रूर	Match key drops nukta; storage keeps it
Same word, two meanings	kal = yesterday or tomorrow	Not a resolver problem: MT uses tense (kal ja raha hoon → tomorrow); gloss shows both senses
English loans	phone, doctor, train	Lexicon loan entries map English spelling ↔ Devanagari (फ़ोन); display keeps the English spelling
Unknown words, names	priyanka, gurgaon	IndicXlit top-k, LM-reranked; marked low confidence

End-to-end examples

Learner types	Resolved Devanagari	Canonical shown back	English
mujhe bohot bhook lagi hai	मुझे बहुत भूख लगी है	mujhe **bahut bhookh** lagi hai	I am very hungry
mera phone kharab h	मेरा फ़ोन ख़राब है	mera phone **kharaab hai**	My phone is broken
main kal dilli ja rha hu	मैं कल दिल्ली जा रहा हूँ	main kal dilli ja **raha hoon**	I am going to Delhi tomorrow
main to ghar pe hi hu	मैं तो घर पे ही हूँ	main to ghar pe hi **hoon**	I'm just at home

Bold marks the words whose spelling differs from the learner's input; the app shows these as gentle respelling notes, never as errors.

Learning science

Every learning feature traces to a finding below; features without one are cut or run as experiments.

1. Evidence base

Six findings with strong replication drive the design; each row says what the finding is and the rule it imposes on the product.

Principle	Evidence	Design rule
Retrieval practice	Students who practiced recall kept 61% of a passage after a week versus 40% for those who reread (summary of Roediger & Karpicke, 2006); the effect has a century of replications (Roediger & Karpicke, 2018)	Every exercise asks the learner to produce or recognize an answer; there is no "read the list" mode. New words are tested within the same session they are introduced
Corrective feedback	Retrieval benefits hold even after failed attempts, provided the correct answer is then shown (Testing effect notes)	Every miss immediately shows the correct Devanagari, canonical spelling and audio
Spacing	A meta-analysis of 839 assessments in 317 experiments found the best gap between sessions grows with how long you need to remember (Cepeda et al., 2006)	Reviews are scheduled by a memory model, never in fixed daily blocks
Model-based scheduling	FSRS predicts recall better than SM-2 in the large majority of collections in the open benchmark (Anki forum); Duolingo's trainable half-life regression roughly halved error versus a Leitner system (Duolingo)	Use FSRS (py-fsrs server, ts-fsrs offline); log every review in a form its optimizer can train on
Interleaving	Mixing categories beat blocking for learning to tell them apart, even though learners believed blocking helped more (Kornell & Bjork, 2008, via Birnbaum et al.); benefits are larger when categories are similar (meta-analysis)	Confusable pairs are mixed within a session: त/ट, क/ख, ki/ke/ka, achha/achhi/achhe
Vocabulary coverage	Most learners need to know about 98% of the words in a text to read it unassisted, and 95% for minimal comprehension (Hu & Nation, 2000, via Nation 2006)	Teach words in frequency order; reading passages are served only when the learner knows at least 95% of their tokens

Pronunciation needs its own method

Adult English speakers struggle to hear the Hindi dental–retroflex contrast; computer training produced about 10% improvement with little transfer to new speakers (Pruitt, 1993). Difficulty also varies by talker and listener (OSF study), and overnight sleep helps consolidate the learning (Earle et al., 2025).

Rules: use several voices for every minimal pair; give feedback on each trial; spread perception practice across days; set expectations that this contrast takes weeks, and never block progress on it.

Setting expectations

The US Foreign Service Institute estimates about 1,100 classroom hours for an English speaker to reach professional proficiency in Hindi (FSI rankings). The app's honest goal is everyday conversation (CEFR A1–A2) plus script literacy, and it says so.

2. Learner model

Knowing a word is several separate skills, so each lemma spawns up to five cards, each scheduled independently by FSRS and unlocked in order.

Card type	Prompt → answer	Unlocked when
Meaning recognition	HV-Roman + audio → English (type or pick)	Word introduced
Meaning production	English → type Hindi	Recognition reaches Review state
Listening	Audio only → type Hindi	Production reaches Review state
Script reading	Devanagari → type HV-Roman	Learner's script stage covers all the word's letters
Usage (cloze)	Sentence with a gap → inflected form	Word's grammar unit is completed

Sibling burying: two cards of the same lemma are never shown on the same day, so one does not give away the other.

Automatic grading → FSRS rating

FSRS expects four ratings. The grader derives them from what the resolver did and how long the learner took, so learners never rate themselves.

Outcome	Rating
Wrong word, no answer, or "show answer" pressed	Again

Right word reached only through the phonetic key or an edit-distance match, or a hint was used	Hard
Right word via canonical or an accepted variant	Good
Canonical spelling and response time under the learner's own 25th-percentile time for that card type	Easy

For production cards, a correct lemma in the wrong inflection (achha for achhi) is rated Hard with a targeted grammar note, not Again. This mapping is a hypothesis: check it in Phase 3 by confirming FSRS's predicted recall is well calibrated against observed recall on these ratings.

Scheduler settings

- Desired retention: default 0.90, adjustable 0.80–0.95 in settings.
- Parameters: global defaults until a learner has enough reviews, then per-learner optimization with the py-fsrs Optimizer (py-fsrs), rerun weekly.
- Learning steps for new cards: 1 minute and 10 minutes within the session (py-fsrs defaults), so new words are retrieved twice before the session ends.

Daily session

1. Due reviews first, capped at 100 a day so a missed week does not become a wall.
2. New items: default 10 a day, mixed into the reviews rather than shown as a block.
3. A short interleaved perception set (minimal pairs) every second session.
4. Typical length 10–15 minutes; the session ends when due cards are cleared, not on a timer.

3. Curriculum

Two levels ship first: A1 (about 500 lemmas plus survival phrases) and A2 (to about 1,500 lemmas). Words enter in frequency order within themed units, and grammar is taught through the words already known.

Word list construction. Rank lemmas by frequency (wordfreq's Hindi list, which draws on several corpora, cross-checked against Dakshina's Hindi Wikipedia text) (wordfreq). Remove proper nouns and Sanskrit-register words rare in speech, add high-frequency spoken words that written corpora under-count (haan, achha, yaar, theek), and have a native speaker review the final list.

A1 units

#	Unit	Grammar target	Example sentence
1	Greetings and politeness	namaste, dhanyavaad, aap as default	aap kaise hain?
2	Me and you	Pronouns + hona: hoon, hai, ho, hain	main student hoon
3	Things around me	Noun gender; ye / vo	ye kitaab hai
4	Describing	Adjective agreement -aa / -i / -e	achhi chaay
5	Numbers, money, time	Numerals 1–100, kitna / kitne	ye kitne ka hai?
6	Where things are	Postpositions + oblique: mein, par, ke paas	kitaab mez par hai
7	Whose is it	ka / ki / ke	Ram ki gaadi
8	Daily routine	Habitual present -ta hai	main roz chaay peeta hoon
9	Right now	Progressive raha hai	vo kha rahi hai
10	Asking for things	Imperatives at three politeness levels	paani dijiye
11	Plans	Future -ga / -gi / -ge	main kal aaunga
12	Food and travel phrases	Review, set phrases	mujhe ye chahiye

A2 adds the perfective and ne, compound verbs, wanting and needing (chahiye, chaahna), can and must (sakna, padna), comparatives, and subjunctive requests.

Script bridge

Devanagari is taught alongside, never instead of, the Roman track. Letters are introduced in groups ordered by frequency in the learner's known words, so every new letter immediately unlocks real words to read.

Stage	What the learner sees	Advance when
1. Roman	HV-Roman only	Learner opts in to script (prompted after unit 3)
2. Roman over script	HV-Roman with Devanagari beneath	90% accuracy on the last 50 script-reading cards in the letter group
3. Script over Roman	Devanagari, Roman revealed on tap	Same 90% / 50-card gate, per letter group

4. Script only	Devanagari; Roman only in feedback	Learner's choice once all groups pass stage 3

The 90% and 50-card thresholds are starting values to tune with Phase 3 data.

4. Measuring whether learners learn

Engagement is not learning, so the primary metrics are memory and skill measures computed from review logs and periodic checks, with engagement tracked only as a guardrail.

Metric	Definition	Use
Observed retention	Share of reviews rated Good or Easy, per card type, per week	Should sit near the desired retention (0.90); a large gap means the grading map is miscalibrated
Scheduler calibration	Log loss and binned RMSE of FSRS predicted recall vs outcome, the metrics used by the open benchmark (srsbench)	Validates the automatic grading map
Known words	Lemmas with predicted recall above 0.9 on both recognition and production	Headline progress number shown to learners
Delayed test	Items not reviewed for 30 days, tested once without feedback	The honest measure of durable learning
Script reading speed	Correct words per minute on a fixed graded passage, monthly	Tracks the script bridge
Perception accuracy	Minimal-pair identification on unseen voices	Tracks pronunciation training transfer
Day-7 and day-30 return	Share of new learners active on those days	Guardrail only

Experiments

- Randomize at the learner level, never per session, so one person is not in two arms.
- Write down the primary metric and minimum sample before starting; stop at the planned size.
- First experiments: desired retention 0.85 vs 0.90; interleaved vs blocked minimal pairs; new items 10 vs 15 a day.

Ethics

No streak-loss guilt notifications, no paywalled "streak repair", and no hiding due-review counts to inflate session starts. Learners can export their full review history and delete their account and data.

Technical architecture

Python for the language engine and API, one TypeScript app for web and mobile, and a versioned lexicon file as the shared contract between them.

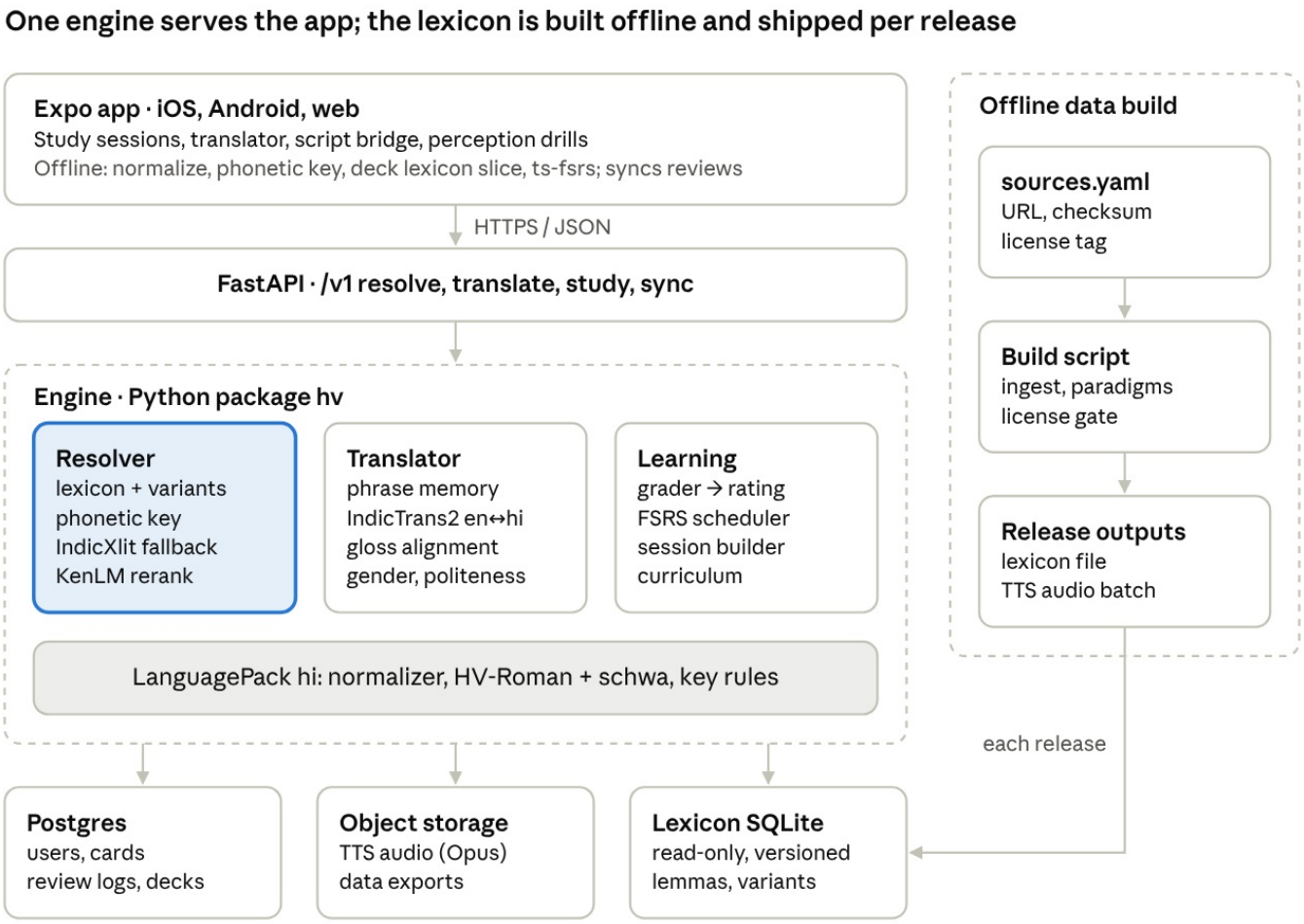
1. Stack decisions

Per the Oct 26 decision, each component starts with the simplest thing that works and names its upgrade, so nothing has to be redesigned to improve it.

Component	Start with	Why	Upgrade path
Engine language	Python 3.12, packaged as `hv`	Every Indic NLP library used here is Python	Port hot paths (normalize, key, lookup) to TypeScript for offline grading
API	FastAPI + Pydantic v2, served by Uvicorn	Typed request/response models generate the OpenAPI spec the app consumes	Split model inference into its own service when load requires
Lexicon store	Read-only SQLite file built offline, versioned, loaded per worker	Fast indexed lookups, one file to ship and diff	Same schema in Postgres if editors need live curation
App data	Postgres 16 via Supabase (also provides auth and file storage)	One managed service covers users, cards, logs and audio	Self-hosted Postgres; keep SQL portable
Translation	IndicTrans2 distilled 200M, both directions, via Hugging Face transformers + IndicTransToolkit (model card; toolkit)	MIT-licensed, strong Hindi quality, CPU-capable	INT8 ONNX or CTranslate2 builds (ONNX packages); 1B model on GPU
Transliteration fallback	`ai4bharat-transliteration` XlitEngine, Hindi, beam 10 with rescoring (PyPI)	Ranked Devanagari candidates for unseen words	CTranslate2 build that drops the Fairseq dependency (HF)
Token language ID	English wordlist + lexicon membership + character n-gram classifier	No model download; handles homographs with context	IndicLID (MIT; has a hin_Latn class) for sentence-level calls (IndicLID)
Sentence reranking	KenLM 5-gram trained on normalized Hindi Wikipedia text from Dakshina	Small, fast, well understood	Small transformer LM
Schwa deletion	Ohala rule + curated lists	Works day one	Arora et al. classifier
Script normalization	Own normalizer following section 4 of the spec	No heavy build tooling	Nisaba (Apache 2.0) via its Python bindings
Scheduler	py-fsrs on server; ts-fsrs on device	Same algorithm both sides	Per-learner parameter optimization
TTS	Indic Parler-TTS, audio pre-generated for every lexicon word and phrase, stored as Opus (model)	Apache 2.0; named Hindi voices give voice variety for perception drills	Native-speaker recordings for the core 500 words
App	Expo (React Native) + TypeScript, one codebase for iOS, Android and web	Matches the web-and-mobile decision with one team	Native modules only if audio latency demands
Fonts	Noto Sans Devanagari, bundled	Consistent conjunct rendering on all platforms	
Hosting	Docker image on a single managed VM or container host	Models load once at startup	Autoscaled container service; GPU node for MT
Observability	Sentry for errors, structured JSON logs, OpenTelemetry traces	Find slow pipeline stages	Dashboards per ambiguity class

Memory planning estimate: a 200M-parameter model in 32-bit floats needs roughly 0.8 GB, so two directions need about 1.6 GB before INT8 quantization. Measure this in Phase 2 rather than trusting the estimate.

2. System diagram



The resolver is highlighted because every other part depends on it: translation, grading and the offline app all start by resolving romanized text to Devanagari. All Hindi-specific rules live in the LanguagePack band, so a second language replaces that band only.

3. Core interfaces

All language-specific behavior sits behind one `LanguagePack` protocol; the resolver, translator and learning code never mention Hindi. Adding a language means writing a new pack, not editing the pipeline.

```
from dataclasses import dataclass, field
from typing import Protocol, Sequence

@dataclass(frozen=True)
class Token:
    text: str            # as typed
    norm: str            # Latin-normalized
    start: int; end: int # offsets into the original input
    lang: str            # "hi" | "en" | "num" | "punct"
    lang_conf: float

@dataclass(frozen=True)
class Candidate:
    native: str          # storage-form Devanagari
    lemma_id: int | None
    canonical: str       # HV-Roman of this surface form
    source: str          # "canonical" | "variant" | "key" | "edit" | "xlit"
    score: float

@dataclass
class Resolution:
    tokens: list[Token]
```

```
        best: list[Candidate | None]     # one per token; None = pass-through
        alternatives: list[list[Candidate]]
        confidence: list[float]

    class LanguagePack(Protocol):
        code: str                        # "hi"
        mt_code: str                     # "hin_Deva" (IndicTrans2 tag)
        scheme_version: str              # "hv-roman 1.0.0"

        def normalize_native(self, s: str) -> str: ...
        def match_key_native(self, s: str) -> str: ...
        def normalize_latin(self, s: str) -> str: ...
        def phonetic_key(self, latin: str) -> str: ...
        def to_canonical(self, native_word: str) -> str: ...
        def to_precise(self, native_word: str) -> str: ...
        def edit_costs(self) -> dict[tuple[str, str], float]: ...
        def lexicon_path(self) -> str: ...
        def tts_voices(self) -> Sequence[str]: ...

    class Resolver:
        def __init__(self, pack: LanguagePack, lexicon, lm, xlit, langid): ...
        def resolve(self, text: str) -> Resolution: ...

    class Translator:
        def to_english(self, res: Resolution) -> "Translation": ...
        def from_english(self, text: str, *, speaker_gender: str,
                         politeness: str) -> "Translation": ...

    class Grader:
        def grade(self, card, answer: str, latency_ms: int) -> "Grade": ...
```

Contract tests run against every pack: normalization is idempotent, `to_canonical` round-trips through the resolver for every lexicon entry, and `phonetic_key` maps every listed variant of a lemma to keys that include the lemma's candidates.

4. API specification

A versioned JSON API under `/v1`; FastAPI generates the OpenAPI document, and the app's TypeScript client is generated from it so the two cannot drift.

Method + path	Purpose	Key fields
POST /v1/resolve	Romanized text → Devanagari candidates per token	text, lang
POST /v1/translate	Either direction, with gloss and respelling notes	text, direction (rom_to_en, en_to_rom), lang, speaker_gender, politeness
POST /v1/romanize	Devanagari → canonical + precise	text
GET /v1/lexicon/{lemma_id}	Entry, forms, glosses, audio URLs	
GET /v1/lexicon/search	Look up by English, any spelling, or Devanagari	q
GET /v1/study/session	Due cards + new items for today	limit_new
POST /v1/study/review	Grade an answer, update FSRS, return feedback	card_id, answer, latency_ms, client_review_id
POST /v1/study/sync	Upload reviews done offline	reviews[] (idempotent by client_review_id)
POST /v1/decks/{deck_id}/items	Save a translated sentence or word to a deck	text, native, source
GET /v1/progress	Known words, retention, script stage	
GET /v1/me/export	Full data export	
DELETE /v1/me	Account and data deletion	

Example: POST /v1/translate

```
    // request
    { "text": "mera phone kharab h", "direction": "rom_to_en", "lang": "hi" }

    // response
    {
      "english": "My phone is broken.",
      "native": "मेरा फ़ोन ख़राब है",
      "canonical": "mera phone kharaab hai",
```

```
          "tokens": [
            { "text": "mera",   "lang": "hi", "native": "मेरा", "canonical": "mera",    "gloss": "my (m.)",   "confidence": 0.97,
"respelled": false },
            { "text": "phone",  "lang": "en", "native": "फ़ोन",  "canonical": "phone",   "gloss": "phone",     "confidence": 0.99,
"respelled": false },
            { "text": "kharab", "lang": "hi", "native": "ख़राब", "canonical": "kharaab", "gloss": "broken, bad", "confidence": 0.93,
"respelled": true },
            { "text": "h",      "lang": "hi", "native": "है",   "canonical": "hai",     "gloss": "is",        "confidence": 0.88,
"respelled": true }
          ],
          "alternatives": [],
          "engine": { "scheme": "hv-roman 1.0.0", "lexicon": "2026.11.0", "mt": "indictrans2-indic-en-dist-200M" }
        }
```


Errors use RFC 9457 problem+json. Every response carries the scheme, lexicon and model versions so any reported mistake can be reproduced exactly.

5. Budgets, offline behavior, security and privacy

These are proposed targets for a single CPU server; Phase 2 measures them and either meets them or revises them in this table.

Operation	p95 target	Where it runs
Romanize a sentence (Devanagari → HV-Roman)	5 ms	Server and device
Resolve a 10-word romanized sentence	50 ms (lexicon path), 250 ms with IndicXlit fallback	Server
Translate a 15-word sentence	800 ms on CPU	Server
Grade a study answer	30 ms	Device (offline) and server
App cold start to first card	2 s on a mid-range phone	Device
Lexicon download for offline use	under 15 MB compressed for A1–A2 words, variants and paradigms	Device

Offline behavior

- Studying works fully offline: the app holds the learner's deck slice of the lexicon, a TypeScript port of normalization, phonetic key and lookup, and ts-fsrs.
- Reviews done offline are queued with a client-generated id and synced idempotently; the server's FSRS replays them in timestamp order and is the source of truth.
- Translation and free-text resolution need the network; the UI says so instead of failing silently.

Security and privacy

- Translation text is not stored by default; review logs store the learner's answer because grading quality depends on it, and this is stated in the privacy notice.
- Learner identifiers in analytics are pseudonymous; raw answers are excluded from analytics exports.
- Export and delete endpoints exist from the first public release.
- Rate limiting on /translate and /resolve per account and per IP, since they run the expensive models.
- If the app may be used by people under 13 (or the local age of digital consent), get legal review before launch; the default plan is an age gate.

6. Extending beyond Hindi

The Hindi design generalizes in two tiers: other Indic languages reuse almost every resource, while other script families reuse the architecture but need new data, new rules and often a different translation model.

What every new LanguagePack must supply

1. A written canonical romanization spec in the format of the Linguistics tab, reviewed by a native speaker and a linguist.
2. A native-script normalizer and match key.
3. A variant source: attested romanizations (lexicon pairs) plus rule-generated alternations.
4. A word frequency list and a monolingual corpus for the language model.
5. A translation model with that language and English.
6. TTS voices, and a gold test set of romanized sentences.

By script family

Family	Examples	Reusable resources	Language-specific work

Other Brahmic (Indic)	Bengali, Marathi, Punjabi, Gujarati, Tamil, Telugu	Dakshina covers 12 South Asian languages (repo); Aksharantar covers 21; IndicXlit, IndicTrans2 (22 languages), IndicLID, Indic Parler-TTS; Nisaba normalization	Schwa behaves differently (Bengali keeps medial schwas Hindi deletes); Marathi shares Devanagari, so the pack needs a language tag, not just a script
Perso-Arabic	Urdu, Sindhi, Kashmiri	Dakshina (Urdu, Sindhi), IndicTrans2	Short vowels usually unwritten, so the canonical romanization needs vowel prediction from the lexicon; right-to-left UI
Cyrillic, Greek	Russian, Ukrainian, Greek	uroman as a baseline candidate generator (uroman)	Mostly alphabetic, so romanization is closer to letter mapping; stress is unwritten
Arabic, Hebrew	Arabic, Hebrew	uroman baseline	Unwritten vowels as for Urdu; Arabic chat alphabet uses digits for letters (3, 7), so the Latin normalizer must keep digits inside words
CJK	Mandarin, Japanese, Korean	Established standards (pinyin, Hepburn, Revised Romanization) can serve as the canonical scheme	Word segmentation; tones (Mandarin) and kanji readings need a dictionary, not character rules

uroman's own documentation says a romanizer is not a full transliterator (uroman); use it to propose candidates and test coverage, never as the taught standard.

Translation outside IndicTrans2's 22 languages needs a separate model choice, to be made when the second family is scoped.

Development plan

Tickets are written so a new developer can pick one up cold: each names its inputs, its output, and the test that proves it is done.

1. Repository and conventions

One monorepo, three deployables (engine+API, app, data build), and the spec tabs exported into `docs/spec` so code review can link to them.

```
hinglishvinglish/
├── engine/                # Python package `hv`
│   ├── hv/core/           # Token, Candidate, Resolution, LanguagePack protocol
│   ├── hv/langs/hi/       # Hindi pack: normalize, scheme, schwa, key, paradigms
│   ├── hv/resolve/        # langid, candidates, rerank
│   ├── hv/translate/      # phrase memory, NMT adapter, gloss
│   ├── hv/learn/          # grader, scheduler wrapper, curriculum
│   └── tests/             # unit, golden, property, contract
├── api/                   # FastAPI app; imports hv
├── app/                   # Expo + TypeScript
│   └── src/offline/       # TS port of normalize, key, lookup; ts-fsrs
├── data-build/            # ingest + build scripts -> lexicon-<ver>.sqlite
│   ├── sources.yaml       # every source, URL, checksum, license tag
│   └── curated/           # hand-maintained overrides, phrases, paradigms (CSV)
├── eval/                  # gold sets, benchmark runners, reports
└── docs/spec/             # exported spec tabs; golden tests generated from tables
```

Area	Convention
Python	uv for environments, ruff for lint and format, mypy strict, pytest + hypothesis
TypeScript	pnpm, ESLint, Vitest, strict mode
CI	GitHub Actions: lint, type-check, unit and golden tests on every PR; gold-set evaluation nightly and on any change under langs/ or data-build/
Versioning	Engine and API: semver. HV-Roman scheme: major on any canonical change. Lexicon: calendar version (2026.11.0) plus the scheme version it was built with
Data	Large sources are never committed; `sources.yaml` pins URL and SHA-256, and the build refuses a checksum mismatch
Definition of done	Code, tests, updated spec tab if behavior changed, and the eval report shows no regression on any ambiguity class
Curated data edits	Pull requests to CSVs in `curated/`, reviewed by a Hindi-speaking reviewer

2. Phase 0 and Phase 1 backlog

Phase 0 builds the deterministic core and the test harness; Phase 1 makes romanized input resolve reliably. No model downloads are needed until P1-07.

Phase 0: Foundations

ID	Task	Acceptance criteria	Depends on
P0-01	Monorepo scaffold, CI, tooling (Development plan §1)	CI runs lint, types, tests on a PR; a failing test blocks merge	—
P0-02	Core types and LanguagePack protocol (Architecture §3)	mypy strict passes; a stub pack satisfies the contract tests	P0-01
P0-03	Devanagari normalizer + match key (Linguistics §4)	Every row of the §4 table is a passing test; hypothesis test: idempotent on random Devanagari	P0-02
P0-04	Latin normalizer (Linguistics §4)	Same, for Latin; diacritic input folds to ASCII	P0-02
P0-05	Character mapping + context rules (Linguistics §2, §3 context rules)	Golden tests generated from the §2 and §3 tables all pass	P0-03
P0-06	Rule schwa deletion (Linguistics §3)	All §3 examples pass; accuracy measured and reported on Arora et al.'s Hindi data	P0-05
P0-07	sources.yaml + license tags + checksum fetcher	Build fails on checksum mismatch or a source with no license tag	P0-01
P0-	Gold sets: Dakshina Hindi dev/test lexicon and 10k romanized	Files in eval/ with a README stating provenance; test split never used	P0-07

08	sentences, split and frozen	for tuning	
P0-09	Remove Kaggle runtime download from the old app; CLI calls the new engine	`python cli.py` runs offline with no credentials	P0-05

Phase 1: Romanization engine

ID	Task	Acceptance criteria	Depends on
P1-01	Lexicon schema + build script producing lexicon-<ver>.sqlite	Schema matches Plan overview; build is reproducible (same inputs → identical file hash)	P0-07
P1-02	Ingest Dakshina romanization lexicon with attestation counts	Variant rows carry source=dakshina and counts; spot check of 100 rows by a Hindi speaker	P1-01
P1-03	Ingest Aksharantar Hindi pairs	source=aksharantar; duplicates merged with Dakshina, counts summed	P1-01
P1-04	Rule-generated variants (accepted-input column of Linguistics §2)	Every lemma has its canonical + rule variants; no variant maps to more than 50 lemmas without review	P1-01
P1-05	Paradigm generator for regular nouns, adjectives, verbs (Linguistics §6)	Generated forms for 200 test lemmas match a native speaker's list at ≥ 98%	P1-01
P1-06	Phonetic key + weighted edit distance (Linguistics §5)	§5 tables pass; keys indexed in SQLite	P1-01
P1-07	IndicXlit fallback adapter	Returns top-5 Devanagari for unseen words; cached	P0-02
P1-08	Token language ID (homograph-aware)	≥ 95% token accuracy on a 500-sentence hand-labeled code-mixed set (to be built in this ticket)	P1-02
P1-09	Candidate scoring + weight tuning on Dakshina dev	Tuned weights committed; top-1 on Dakshina test beats IndicXlit alone	P1-02–P1-07
P1-10	Arora et al. schwa classifier behind a flag	Beats the rule on held-out words; enabled only if it does	P0-06
P1-11	Round-trip CI invariant	For every lexicon entry, resolving its canonical spelling returns that entry; 100% required	P1-09

Estimate (one engine developer): Phase 0 about 3 weeks, Phase 1 about 4–5 weeks.

3. Phase 2 and Phase 3 backlog

Phase 2 adds translation and the API; Phase 3 is the first learner-facing release, so the app work starts here in parallel.

Phase 2: Translation and API

ID	Task	Acceptance criteria	Depends on
P2-01	KenLM 5-gram on normalized Hindi Wikipedia (Dakshina native text)	Model built reproducibly; reranking improves sentence-level word accuracy on the Dakshina romanized sentences	P1-09
P2-02	Phrase memory: 600 curated A1–A2 phrases written in-house, native-speaker reviewed	CSV in curated/ with English, Devanagari, canonical, register, unit; owned content, no third-party license	P1-01
P2-03	IndicTrans2 adapter, both directions (dist-200M)	Batching, sentence splitting, IndicProcessor pre/post-processing; deterministic decoding (beam 4)	P0-02
P2-04	Translator: phrase memory first, NMT otherwise; gloss alignment	Exact phrase hits never call NMT; every Hindi token gets a lemma gloss	P2-02, P2-03
P2-05	Speaker-gender and politeness handling for English → Hindi	Test set of 100 first-person sentences: 100% use the requested gender; politeness tags respected	P2-04
P2-06	FastAPI service with the §4 Architecture endpoints (resolve, translate, romanize, lexicon)	OpenAPI generated; contract tests; problem+json errors; version fields in every response	P2-04
P2-07	MT evaluation harness	chrF++ on IN22-Conv with Devanagari input vs our romanized-then-resolved input, reported per release	P2-04
P2-08	Latency and memory measurement; ONNX or CTranslate2 conversion if budgets fail	Architecture §5 table updated with measured numbers	P2-06

Phase 3: Learning MVP

ID	Task	Acceptance criteria	Depends

			on
P3-01	Postgres schema (users, cards, review_log, decks) + Supabase auth	Migrations versioned; row-level security on per-user tables	P2-06
P3-02	Grader with the outcome → rating map (Learning science §2)	Table-driven tests for every row of the rating map, including wrong-inflection → Hard	P1-11
P3-03	Scheduler service on py-fsrs; session builder (due first, cap 100, 10 new, interleaved)	Simulated 60-day learner produces sensible due counts; sibling burying enforced	P3-01
P3-04	A1 curriculum content: 12 units, word lists, sentences, cloze items	Native-speaker reviewed; every item resolves through the engine without warnings	P2-02
P3-05	TTS batch: Indic Parler-TTS audio for all A1 items in two voices	Opus files on storage; 5% random sample checked by a native speaker for schwa and nasal errors	P3-04
P3-06	Expo app: onboarding (gender, politeness, goal), study session, five card types, feedback screens	Works on iOS, Android and web builds; screen-reader labels on all controls	P3-03
P3-07	Translator screen with gloss and save-to-deck	Saved sentence becomes cards in the learner's deck	P2-06, P3-06
P3-08	Offline study: TS port of normalize/key/lookup + ts-fsrs + sync	Port passes the same golden tests as Python (shared JSON fixtures); offline reviews sync idempotently	P3-02, P3-06
P3-09	Pilot: 20–50 learners for two weeks	Calibration and retention metrics (Learning science §4) reported; top 20 grading complaints triaged	P3-01–P3-08

Estimate (one engine developer, one app developer): Phase 2 about 4 weeks, Phase 3 about 6 weeks including the pilot.

4. Phase 4 backlog and launch checklist

Phase 4 turns the pilot into a public app: the A2 level, script bridge, pronunciation training, and the operational work a public launch needs.

ID	Task	Acceptance criteria	Depends on
P4-01	A2 curriculum (to about 1,500 lemmas): perfective + ne, compound verbs, modals	Native-speaker reviewed; same engine checks as P3-04	P3-09
P4-02	Script bridge, four stages with per-letter-group gates (Learning science §3)	Gate thresholds configurable; stage changes logged	P3-06
P4-03	Minimal-pair perception trainer (त/ट, द/ड, aspirated pairs, nasal vowels)	At least four voices per pair; feedback each trial; unseen-voice test set held out	P3-05
P4-04	Native recordings for the core 500 words, replacing TTS	Recording guide, consent forms, loudness-normalized files	P3-05
P4-05	Per-learner FSRS optimization job (weekly)	Runs only above a minimum review count; log loss not worse than global parameters, else keep global	P3-03
P4-06	Progress dashboard: known words, retention, script stage, delayed-test results	Numbers match server-side metric definitions exactly	P3-09
P4-07	Graded reading passages served at ≥ 95% known-token coverage	Coverage computed from the learner's known-word set at request time	P4-01
P4-08	Experiment framework (learner-level randomization, pre-registered metric)	First experiment from Learning science §4 running	P4-06
P4-09	Account export and deletion, privacy notice, age gate	End-to-end tested; deletion removes rows and audio uploads	P3-01

Launch checklist

- ☐ Data & licensing tab: every source in the release build is tagged commercial-OK, or the build is explicitly non-commercial
- ☐ Round-trip invariant at 100% for the shipped lexicon
- ☐ No regression on any ambiguity-class test set versus the pilot build
- ☐ Measured latency meets the Architecture §5 budgets, or the budgets were revised with sign-off
- ☐ Native-speaker review of all A1–A2 content and a 5% audio sample
- ☐ Crash-free sessions above 99.5% in the final beta week
- ☐ Privacy notice, export, deletion and age gate live
- ☐ Store listings, accessibility review (screen reader, dynamic type) done

Estimate: Phase 4 about 8 weeks with the two-developer team plus a part-time Hindi content reviewer, so roughly 25–27 weeks from Phase 0 to public launch.

5. Testing strategy

Seven layers, from milliseconds to weeks; the first four run on every pull request and the gold-set layer gates merges that touch language code or data.

Layer	What it checks	Data	Runs
Unit	Each function's documented behavior	Inline cases	Every PR
Golden	Spec tables in the Linguistics tab	JSON fixtures generated from docs/spec, shared by Python and the TypeScript port	Every PR
Property	Normalization idempotent; canonical round-trip; key stable under case and repeat-letter noise	hypothesis-generated strings and lexicon entries	Every PR
Contract	Every LanguagePack satisfies the protocol; API matches OpenAPI	Stub and real packs	Every PR
Gold set	Top-1 word accuracy, sentence word accuracy, per ambiguity class	Dakshina Hindi test split, hand-labeled code-mixed set, ambiguity-class sets	Nightly + PRs touching langs/ or data-build/
MT quality	chrF++ (sacrebleu) on IN22-Conv: Devanagari input vs romanized input	IN22-Conv	Per release
Human	Adequacy and fluency of translations and respellings	200 sampled sentences per release	Per release

Human evaluation protocol

1. Sample 200 sentences: 100 from real (consented) learner inputs, 100 from IN22-Conv romanized by a native speaker.
2. Two bilingual raters score each output 1–5 for adequacy and fluency, blind to system version.
3. Report mean scores and inter-rater agreement; disagreements of two or more points go to a third rater.
4. Any release whose adequacy drops by 0.2 or more versus the previous release is blocked pending review.

Gold-set hygiene

The Dakshina test split and the code-mixed set are frozen at P0-08 and never used for tuning or curated-list decisions. When a mistake from the test set is fixed by a curated override, the item moves to a separate "seen" file and is no longer counted as test.

Data and licensing

Functionality comes first, but every source is tagged now so a commercial build can be produced later without an audit.

1. Provenance and licenses

Most of the stack is usable commercially; the two real constraints are Dakshina's share-alike clause and the IIT Bombay corpus, which must stay out of any commercial build. "Verify" means the license was not confirmed from a primary page during this research.

Source	Used for	License (as found)	Commercial	Obligations / notes
Dakshina	Variant lexicon, gold sets, Hindi Wikipedia text for the LM	CC BY-SA 4.0 (via FlipRate summary)	Yes	Attribution; a distributed lexicon that includes Dakshina-derived rows must also be CC BY-SA. Keep those rows tagged so they can be split out
Aksharantar	Variant lexicon	Manually collected data CC-BY; mined data and existing sources CC0	Yes	Attribution for the manual portion. One older dataset-card revision was tagged cc-by-nc-4.0; verify the current card before a commercial build
IndicTrans2 models	Translation	MIT	Yes	Hugging Face repo is gated: accept the conditions with the project account
IndicXlit models	Transliteration fallback	Verify: an older PyPI release stated the models are CC BY-SA 4.0 (PyPI 0.5.0.3)	Likely, with share-alike	Model outputs are probably not derivatives, but get legal review
IndicLID	Sentence language ID	MIT	Yes	
Indic Parler-TTS	Audio	Apache 2.0 (summary)	Yes	Gated repo; check gating terms
Nisaba	Script normalization (upgrade)	Apache 2.0	Yes	
py-fsrs	Scheduling	MIT	Yes	ts-fsrs license: verify
wordfreq	Frequency ranking	Verify code and data licenses	Likely	
indic_transliteration	Precise (diacritic) view, tests	Verify	Likely	
uroman	Future non-Indic baselines	Verify	Unknown	Not used for Hindi
IIT Bombay corpus	Research and evaluation only	Non-commercial research	**No**	Never in a release build
IN22 benchmarks	Evaluation only	Released with IndicTrans2; verify	Eval use	Never shipped
Kaggle English-Hindi	Nothing (dropped)	Not stated on the page	**No**	Removed in P0-09
In-house phrases, curriculum, recordings	Phrase memory, lessons, audio	Owned	Yes	Contributor agreements and speaker consent forms

2. License gate and pre-commercial checks

The build script, not a person's memory, decides what goes into a release: each source in `sources.yaml` carries a license tag, and the build profile declares which tags it accepts.

```
# data-build/sources.yaml (excerpt)
- id: dakshina-hi
  url: https://storage.googleapis.com/gresearch/dakshina/dakshina_dataset_v1.0.tar
  sha256: <pinned>
  license: CC-BY-SA-4.0
  tags: [commercial-ok, share-alike, attribution]
- id: iitb-en-hi
  license: research-noncommercial
  tags: [noncommercial]

# data-build/profiles.yaml
dev:        { accept: [commercial-ok, noncommercial] }
commercial: { accept: [commercial-ok], require_attribution_file: true }
```

- Every lexicon row stores its `source`, so a build profile can exclude a source and the build reports exactly which rows and words were lost.

- The commercial profile writes an ATTRIBUTION file listing every CC-BY and CC-BY-SA source, bundled with the app.
- CI builds both profiles on every data change, so a commercial build is always one command away.

Before the first commercial release

- [ ] Confirm the current Aksharantar dataset-card license on Hugging Face
- [ ] Confirm the IndicXlit model weight license and get legal advice on share-alike for model outputs
- [ ] Legal advice on whether serving lookups from a CC BY-SA-derived lexicon counts as distribution
- [ ] Read and record the gating terms of IndicTrans2 and Indic Parler-TTS repos
- [ ] Confirm wordfreq, ts-fsrs and indic_transliteration licenses
- [ ] Contributor agreements for curriculum writers; consent forms for voice recordings